BALANCE: Bit and Layer-Aware Lightweight ECC Design Method for In-Flash Computing Based LLM Inference Accelerator

Changwei Yan†, Lishuo Deng†, Mingbo Hao, Cai Li and Weiwei Shan*
National ASIC System Engineering Research Center, Southeast University, Nanjing, China
emails: {yancw, dengls, wwshan}@seu.edu.cn; *Corresponding author; †Equal contribution

2026 31st Asia and South Pacific Design Automation Conference (ASP-DAC) | 979-8-3315-9123-6/26/$31.00 ©2026 IEEE | DOI: 10.1109/ASP-DAC66049.2026.11420589

***Abstract*—The shift of Large Language Model (LLM) inference to edge devices demands efficient hardware solutions that overcome memory, power, and computational constraints. In-Flash Computing (IFC) using NAND flash provides an energy-efficient alternative but requires robust Error Correction Codes (ECC), severely limiting logic area and power budgets. This paper presents BALANCE, a lightweight ECC co-design strategy tailored for IFC-based LLM accelerators. BALANCE introduces (1) a fine-grained bit-level significance evaluation, (2) a layer-level sensitivity assessment leveraging cosine similarity of residual connections, and (3) a stratified ECC allocation algorithm optimizing flash memory placement. Experimental evaluations on LLaMA3-8B demonstrate that, compared to state-of-the-art solutions Lincoln and AiF, BALANCE reduces ECC overhead, achieving area savings of 275× and 62×, and power reductions of 899× and 321×, respectively. This is accomplished while delivering 6% and 5.7% higher code rates and maintaining accuracy degradation within 0.5%. To our knowledge, BALANCE is the first work to systematically integrate LLM error sensitivity with NAND flash physics, enabling a new class of highly efficient and powerful IFC accelerators for the edge.**

Keywords—In-Flash Computing, Large Language Model Accelerator, Error Correction Code, NAND flash errors

I. INTRODUCTION

As Large Language Models (LLMs) enter the inference era, model training will remain in the cloud, but inference will increasingly shift to the edge, enhancing accessibility, customization, security, and efficiency [1], [2], [3]. However, traditional edge hardware accelerators face the triple challenges of memory, power, and computational walls. Additionally, single-batch operation in mainstream edge LLMs exacerbates memory bandwidth bottlenecks, limiting their deployment. Existing DRAM-based near-memory computing solutions are still hindered by high storage costs, limiting broad adoption [4], [5]. NAND Flash-based near-memory accelerators leverage low-cost, high-capacity storage and high internal bandwidth of in-flash computing (IFC) to reduce data transfer energy and latency [6], [7], [8]. However, to ensure the reliability of weight data stored in flash, this architecture requires integrating substantial Error-Correcting Code (ECC) circuits into the logic die stacked with a 3D NAND Flash array (Fig. 1). This integration is constrained by limited area and thermal budgets, limiting the number of processing elements (PEs) that can be deployed, which in turn impacts computational power. State-of-the-art (SoTA) IFC LLM accelerators aim to deploy lightweight ECC schemes directly onto the logic die guided by the characteristics of LLM weights [6] and the reliability characteristics of NAND Flash memory [7], [8].

In this work, we further exploit the intrinsic error resilience of LLM weights at a fine granularity and propose BALANCE, a lightweight ECC co-design scheme for IFC-LLM inference chips. This scheme aims to meet the accuracy requirements of LLM inference with minimal ECC overhead. Our main contributions are as follows:

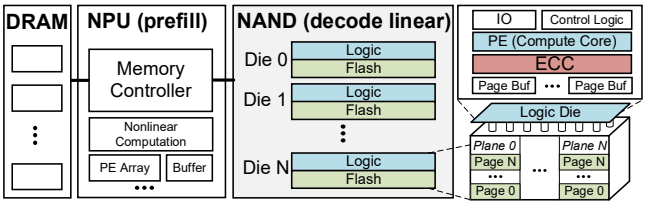

Fig. 1. Typical 3D NAND Flash IFC LLM accelerator architecture [7].

- A comprehensive bit significance evaluation method considering both the numerical weight of each bit and the asymmetric impact of error flips across different model weight data formats.
- A fine-grained layer significance evaluation method leveraging the residual connection structure in LLM decoder layers to calculate inter-layer cosine similarity.
- A stratified ECC protection strategy based on bit-level and layer-level significance ranking, utilizing a greedy algorithm to achieve lightweight protection with Hamming codes.
- A corresponding data placement method that considers leveraging flash multi-plane parallelism and the error rate differences in multi-level cells (MLC).

By employing BALANCE, we strategically protect critical weights, achieving high-precision LLM inference even with minimal protection overhead. Compared to SoTA solutions [7], [8], our approach, when applied to Llama3-8B, reduces ECC circuit area by 275× and 62× compared to Lincoln and AiF, respectively, and lowers power consumption by 899× and 321×, respectively. Additionally, it improves the code rate (effective data ratio) by 6% and 5.7%, while maintaining LLM inference accuracy degradation within 0.5%. Experimental results demonstrate that our method exhibits exceptional adaptability across both multi-level cell (MLC) and single-level cell (SLC) flash memory, delivering an efficient and reliable solution for LLM inference on edge devices. To the best of our knowledge, this is the first work to integrate LLM bit-level and layer-level significance with flash memory physical properties, enabling a low-overhead Hamming code implementation for IFC-based LLM inference accelerators.

II. BACKGROUND

A. LLM Inference and In-Flash Computing

LLMs typically adopt multi-layer transformer decoders (Fig. 2), where each layer includes a self-attention module and a feed-forward network (FFN) wrapped by residual connections. These structures can reach hundreds of billions of parameters, mainly concentrated in Query/Key/Value (Q/K/V) generation, FFN, and projection operations. Matrix-

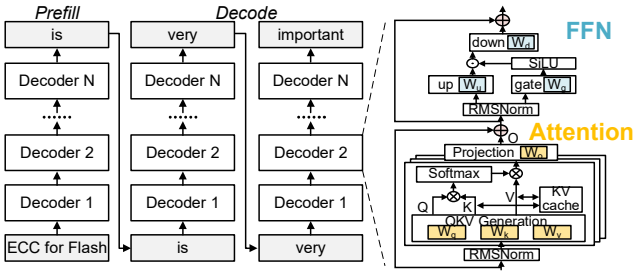

Fig. 2. Typical LLM architecture and execution flow.

matrix and matrix-vector multiplications (GEMMs and GEMVs) constitute the bulk of the computational workload, reflecting the central role [9].

LLM inference consists of two stages: prefill and decode. In prefill, all input tokens are processed in parallel using GEMMs, and the resulting Key/Value vectors are stored in the KV cache. During decode, tokens are generated sequentially, each triggering a GEMV-based FC computation. Due to the low arithmetic intensity of GEMV operations in single-batch LLM inference, computation becomes memory-bound and bandwidth-limited [6], [7], [10]. To address this challenge, Processing-In-Memory (PIM) techniques have emerged as a promising direction. Among them, on-die IFC, which leverages the low cost and high storage density of 3D NAND Flash, offers a compelling solution for near-data processing [2], [11]. Recent advancements have demonstrated impressive performance, achieving up to 36.34 tokens/s for a 7B model [6] and 11.49 tokens/s for a 65B model [7].

As illustrated in Fig. 1, SoTA flash-based PIM dies for LLM acceleration are typically composed of two vertically integrated components: a flash die and a logic die. The flash layer provides high-density, non-volatile storage and supports multi-plane operations to enhance internal parallelism. Stacked beneath or above the flash array, the logic die, fabricated with advanced CMOS technology, contains lightweight compute units, ECC circuitry, and control logic. This tight integration, often achieved via Xtacking [12] or other hybrid bonding techniques [13], enables high-bandwidth, low-latency data exchange between storage and compute, thereby substantially mitigating the data movement bottlenecks inherent in conventional architectures. During inference, weights are first read into page buffers, corrected by lightweight ECC engines, and then fed to on-die PEs for GEMV computation.

B. *Reliability of NAND Flash and SoTA Solutions*

Despite its advantages in density and cost, modern 3D NAND Flash memory suffers from reliability challenges [14], [15], [16], [17], [18]. This phenomenon is exacerbated by the accumulation of program/erase (P/E) cycles and read cycles [19]. Single error detection/correction codes, such as Hamming codes, were previously sufficient to enhance the reliability of SLC Flash [20]. In modern 3D TLC (triple-level cell) NAND devices, the raw bit error rate (RBER) can reach up to 1×10^{-4} after several hours of retention and may exceed 1×10^{-2} with aging [21]. Maintaining system-level reliability requires keeping the uncorrectable BER (UBER) below 10^{-15}, a threshold that becomes increasingly difficult to meet as RBER rises. To combat this, SSDs typically employ advanced Low-Density Parity-Check (LDPC) error correction codes, often leveraging soft-decision decoding to tolerate high RBER values [16]. LDPC codes offer strong error correction but incur high computational and area costs. As a result, they are typically implemented in the off-die flash controller. This architectural separation becomes a major limitation for on-die IFC designs, where only lightweight logic is available and off-chip correction is not feasible.

To address these challenges, recent solutions like Lincoln [7] and AiF [8] manage the raw bit error rate (RBER) of SLC and TLC Flash, respectively, enabling the use of simpler Bose–Chaudhuri–Hocquenghem (BCH) codes. However, this approach remains costly: the BCH circuits in Lincoln and AiF still consume 68% and 80% of the logic die area, respectively, severely limiting the computational PE array.

III. Bit-level and Layer-level Error Sensitivity

A. *Error Injection Framework*

To evaluate the robustness of LLM inference, we conduct error injection experiments on LLM weights. Specifically, we develop a weight error injection framework using PyTorch, which enables controlled bit-flip errors in each layer and bit of weights. For the benchmark, we select LLaMA3-8B [22], OPT-30B [23], and LLaMA3-8B-INT8 [24], representing three distinct numerical formats. Inference performance after error injection is evaluated using the Wikitext-2 dataset [25] for language modeling and the MMLU [26] benchmark for assessing general knowledge and reasoning capabilities. All results are averaged over 50 random error injection trials, and the entire evaluation is conducted on A800 hardware.

B. *Insights and Motivation*

We first conducted error injection experiments on the LLaMA3-8B model. As shown in Fig. 3(a), for the LLaMA3-8B model, correct inference accuracy is maintained only when the error rate is below 10^{-10}. To meet this UBER requirement, high-complexity and overhead-intensive error correction schemes, like BCH or even LDPC, must be employed. As discussed in Section II, these codes occupy the vast majority of the logic die's area. Therefore, our motivation is to minimize ECC area and power while ensuring accuracy.

To implement lower overhead ECC by exploring the feasibility of using Hamming codes, we analyze the weight data in typical LLMs. Common weight data formats for LLMs include BF16 (used in the LLaMA3-8B family), FP16 (used in the OPT family), and quantized formats such as INT8 and INT4 (Fig. 4). The range of values in actual LLM weights generally does not require the higher exponent bits of BF16, making it less efficient. In the LLaMA3 model, both the upper two bits of the exponent are fixed, meaning there is no need to store these bits in flash memory. Due to the distribution characteristics of the weight values, the impact of different bit positions on model accuracy varies significantly. For instance, the high-order exponent bits have a greater influence on accuracy, whereas the lower-order mantissa bits have a relatively smaller impact. This phenomenon presents an optimization opportunity. By eliminating the fixed high-order exponent bits, the tolerated error rate can significantly increase to 10^{-6} (Fig. 3(b)). We further conduct fault injection experiments on two additional models, as shown in Fig. 5. For models quantized to INT8, we found that their error tolerance is significantly higher than that of BF16 or FP16 models, although there are no fixed bits (Fig. 5(a)). This is because in integer formats, even when the highest bit is flipped, the value change is limited to 2^7 or 2^3, respectively. Through error injection after fixing high-order exponent bits, we identified the turning points for accuracy degradation in various models:

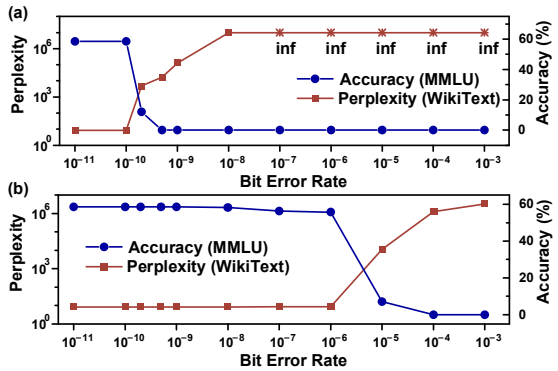

Fig. 3. The error sensitivity test of LLaMA3-8B under (a) uniform bit-flip error and (b) bit-flip error after fixing high-order exponent bits.

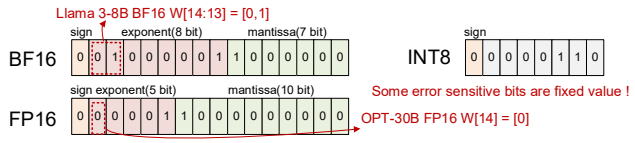

Fig. 4. Data formats of LLaMA3 and OPT model.

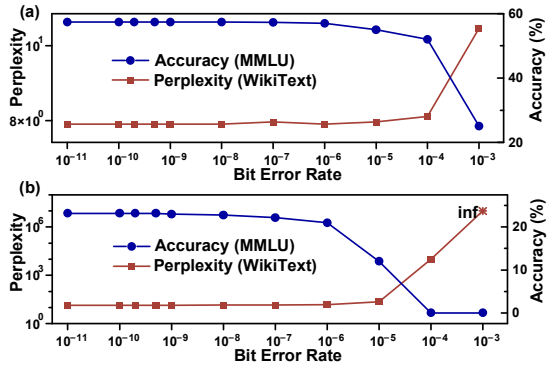

Fig. 5. The error sensitivity test of (a) LLaMA3-8B-INT8 and (b) OPT-30B under bit-flip error after fixing high-order exponent bits in flash.

5×10^{-7} for LLaMA3-8B, 1×10^{-7} for OPT-30B, and 3×10^{-5} for the INT8 version of LLaMA3-8B. Therefore, the LLM weight data have the potential to meet the error correction level of Hamming codes [20], even under the higher BER in TLC.

Through the analysis of the LLM weights, we identify an opportunity to replace complex ECC with the lower-cost Hamming codes. However, the weaker error-correction capability of Hamming codes requires a lower code rate (information bits/total transmission bits) to meet the required UBER. Taking the Single Error Correction (SEC) Hamming code for every 16 bits or 32 bits as an example, these strategies incur additional ECC coding overheads of 23.8% and 15.7%, respectively, which pose significant challenges to the flash storage capacity and bandwidth. Thus, the second step of our optimization goal is to reduce the number of encoding bits required by ECC.

To reduce ECC overhead while maintaining inference accuracy, we propose a stratified ECC scheme based on the error sensitivity of weight data. This two-level analysis considers both bit-level and layer-level significance, where more error-sensitive data are deemed more important due to their greater impact on inference results.

C. Bit significance evaluation method

We begin by performing bit significance evaluation based on the data format. For floating-point representations, bit flips in the mantissa typically result in minor numerical deviations.

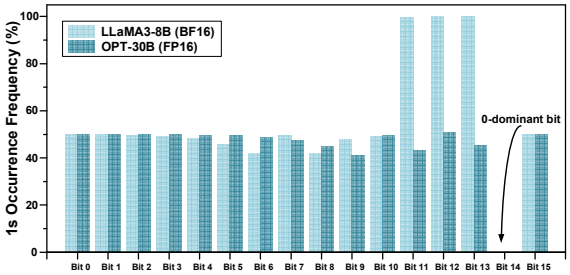

Fig. 6. 1's occurrence frequency in each bit position.

TABLE I. SCORES AND PERPLEXITY UNDER 10^{-3} BER (LLAMA3-8B)

	$S_{bit}(i)$	$S_{bit}(i)$ Rank	Perplexity	Perplexity Rank
Bit7	1.01	8	8.32	8
Bit8	2.32	5	8.47	5
Bit9	8.36	4	568.56	4
Bit10	130.25	2	862626.70	2
Bit11	110.25	3	1602.09	3
Bit12	3.12	7	8.34	7
Bit13	0	9	8.32	9
Bit14	Inf	1	Inf	1
Bit15	2.0	6	8.39	6

Changes in the higher-order bits of the exponent have a greater impact on the result than changes in the lower-order bits. A flip in the sign bit reverses the polarity of the value, with the impact depending on the magnitude of the original weight. For integer data, both the sign bit and data bit flips affect the value by powers of two, and their importance can be directly ranked from the highest to the lowest bit.

Moreover, bit flips from 0 to 1 and from 1 to 0 can lead to significantly different consequences. For instance, flipping an exponent bit from 0 to 1 can cause a substantial increase in the absolute value and create unexpected large outliers, severely affecting computation results. In contrast, flipping from 1 to 0 usually drives the weight toward zero, which many models inherently tolerate due to activation sparsity and weight redundancy, resulting in a less critical impact. Taking the error injection evaluation experiment on the LLaMA3-8B model as an example, under 10^{-2} RBER, the model perplexity of 0-to-1 flips is significantly greater (227k×) than that of 1-to-0 flips. Furthermore, we observe that if a specific bit position in all model weights predominantly holds the value 0, we refer to it as a 0-dominant bit (Fig. 6). Errors injected into such bits (e.g., bit14) result in more pronounced accuracy degradation. Conversely, certain bits (e.g., bit 13), even if they are high-order exponent bits, are 1-dominant and thus experience minimal impact on inference accuracy when bit flips occur.

Based on the above observations, we propose a series of formulas to quantitatively characterize bit importance. Taking the BF16 format as an example, we model the bit-level significance using a scoring function as (1) and (2). Specifically, we define $S_{bit}(i)$ as the bit significance score for bit position i and $P_0(i)$ as the proportion of zeros at bit position i. The scoring captures each bit's contribution to numerical variation and its impact on model performance under error injection. For mantissa bits or integer type, we adopt a simpler method by ranking them in descending order of significance, based on their positional weight in the binary representation (i.e., higher-order bits are assigned higher importance).

$$S_{bit}(i) = P_0(i)\times 2^{2^{i-7}},\ 7 \le i \le 14, (\text{exponent}) \qquad (1)$$

$$S_{\text{bit}}(i) = 2, i = 15, (\text{sign}) \quad (2)$$

We perform random bit-flip injection to derive empirical bit importance rankings (Table I). The results align well with perplexity on the WikiText-2 dataset, validating the method's effectiveness in quickly and accurately quantifying bit significance. The scoring captures both the numerical weight of each bit and the asymmetric impact of 0-to-1 flips, and generalizes well to other data formats.

D. Layer significance evaluation method

As described in Section II.A, modern LLMs consist of stacked Transformer decoder blocks, each comprising attention and FFN layers connected via residual paths. The inter-layer transformation follows the residual form:

$$x^{(l+1)} = x^{(l)} + f\left(x^{(l)}; \theta^{(l)}\right) \quad (3)$$

where $x^{(l+1)}$ represents the input hidden state at layer l, and $\theta^{(l)}$ denotes the parameters of that layer. A residual connection layer at layer l contributes to the transformation $f(x^{(l+1)};\theta^{(l)})$.

Inspired by recent research [27], we further evaluate the importance of each layer in the LLM by measuring the cosine similarity between the inputs and outputs of the attention and FFN layers in each decoder block. Cosine similarity quantifies how closely two vectors align. A higher similarity between a layer's input and output indicates lower importance, as it suggests minimal transformation and limited contribution to the final model output.

We sample from the pre-trained dataset D and record the input and output hidden states of each residual connection layer. The cosine similarity between the input and output of consecutive layers is calculated as:

$$C = cos(x^{(l)}, x^{(l+1)}) = E_{(x_i^{(l)}, x_i^{(l+1)}) \in D}\left(\frac{1}{L}\sum_{j=1}^{L}\frac{x_{i,j}^{(l)} \cdot x_{i,j}^{(l+1)}}{\left\|x_{i,j}^{(l)}\right\| \cdot \left\|x_{i,j}^{(l+1)}\right\|}\right) \quad (4)$$

where D denotes the recorded hidden states from different samples, $x_i^{(l)}, x_i^{(l+1)} \in R^{d \times L}$ denotes the input and output hidden states of the i-th sample, respectively, d denotes the hidden size and L is the sequence length, E represents the average cosine similarity calculated for each sample in D. To assess the importance of each layer, we define the layer significance score $S_{\text{layer}} = 1 - C$. A higher cosine similarity corresponds to a lower layer importance.

We randomly inject errors into each layer of LLaMA3-8B, then evaluate the model's performance and compare the results with the S_{layer} scores ranking (Fig. 7). The results confirm that the proposed method aligns well with observed performance, validating the use of cosine similarity for layer importance estimation. Experimentally, FFN layers in the first and last decoder blocks show higher importance, while those in the middle contribute less. Attention layers follow a similar trend, with early layers being more influential.

IV. Stratified ECC Unit and Flash Deployment

Building on our bit- and layer-level significance analysis, we observe substantial variation in error tolerance across LLM weights. To exploit this, we propose a greedy ECC optimization algorithm that combines weight fault tolerance modeling with the flash device properties. By selectively applying Hamming code protection and optimizing data layout, the method offers a lightweight ECC solution for IFC-based edge LLM accelerators.

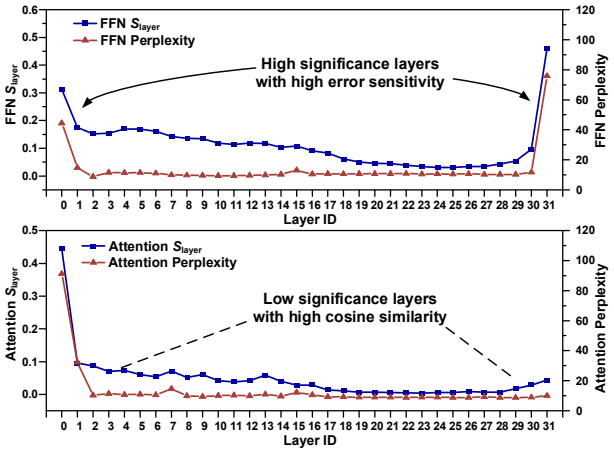

Fig. 7. S_{layer} and perplexity across each FFN and attention layer.

The algorithm proceeds in three stages: (1) simulate errors to define the required protection range, (2) partition weights into stratified ECC units (SEUs) by significance, and (3) iteratively enhance protection for critical SEUs until performance targets are met. This greedy algorithm efficiently addresses the complex weight protection problem through locally optimal decisions. Compared to global search methods, it offers significantly lower time complexity while achieving near-optimal results in practice.

A. Algorithm implementation

The algorithm takes as input the LLM weight data, flash layer configurations (e.g., number of planes, cell type), and target performance constraints. It outputs a weight-to-storage mapping with ECC protection levels across planes, pages, and sub-pages in XLC Flash. The detailed steps are as follows:

1) Performance benchmark and error analysis

First, we test the original LLM model without any error interference and record its accuracy and perplexity as baseline performance metrics, which will serve as a reference standard for optimization. Then, we simulate random bit-flip errors in NAND Flash and analyze the trend of model performance changes at different error rates, thereby determining the maximum tolerable error range. Based on the error tolerance range, we set the maximum level of ECC protection as the starting point for subsequent optimization. For Hamming codes, common protection levels range from typical {8, 16, 32, 64, 128, 256, 512, 1024} bits for single-bit error correction.

2) Stratified ECC unit (SEU) partitioning and ranking

As described in Fig. 8, we perform partitioning and ranking of the weights based on layer and bit significance defined in Section III. Each layer is considered a unit of granularity, and every t-bit forms another unit of granularity. For floating-point format data, we observe a significant difference in importance between the top-3 bits and the remaining bits (Table I), thus, we set $t = 3$. Similarly, we set $t = 2$ for the integer format. In this way, we can partition the weight data into groups indexed by layer and bit group. For the t-bits corresponding to all weight data of the same layer, we define them as the stratified ECC units (SEUs). SEUs can be ranked in two ways: layer-prioritized (purple indices) or bit-prioritized (red indices). Based on empirical results, we adopt bit-prioritized ranking as it yields better performance.

3) ECC Allocation Method

Algorithm 1: Greedy algorithm

Input: SEU list S with importance ranking, Performance threshold T
Output: ECC protection configuration for each SEU

Step 1: Initialize all SEU without protection
for each SEU $s \in S$**:**
  Set protection(s) = None
  performance_met = False
Step 2: Greedy allocation by SEU importance
Sort S by descending order of importance
current_protected = ∅
while not ***performance_met*****:**
  s = the most important remaining element in S
  Add ECC protection to s
  current_protected = *current_protected* ∪ {s}
/* Evaluate model performance */
  ppl = CalculateWikitext2Perplexity()
  mmlu = CalculateMMLUScore()
  if (*ppl* - *original_ppl*) ≤ *T_PPL* and (*original_mmlu* - *mmlu*) ≤ *T_MMLU*:
  performance_met = True
endwhile
return current_protected

To minimize the number of SEUs protected while maintaining inference accuracy, we allocate ECC protection with varying strengths according to the SEU importance ranking. Given hardware overhead considerations, we classify ECC protection into two cases: a) No protection is applied to the weights. b) Protection is applied based on the strength determined in step 1) using the baseline model.

As shown in Algorithm 1, ECC protection is allocated to each SEU following these steps: Initially, no protection is applied to any SEU. ECC protection is then progressively applied to the SEUs with higher importance, one by one, until the model performance metrics meet the required thresholds. Here, model performance is evaluated based on the WikiText2 perplexity and MMLU dataset scores, ensuring that the scores do not exceed the threshold compared to the original model. Once the iteration concludes, each SEU will have a corresponding level of protection.

Finally, to maximize the plane-level parallelism within the flash and thus enhance read bandwidth to optimize inference performance, all the SEUs with completed ECC protection are evenly distributed across different planes (Fig. 9 left).

Our method naturally extends to multi-level cell (XLC) Flash, common in PIM-based LLM accelerators [6]. Taking TLC Flash as an example (Fig. 9, right), each cell stores LSB, CSB, and MSB, forming three sub-pages (lower, middle, upper). These sub-pages exhibit increasing BER from LSB to MSB due to physical device characteristics [28], [29]. After ranking SEUs, we assign the most critical ones to lower-BER sub-pages. For TLC, SEUs are divided into three groups, with the top third placed in the lower page. Before ECC optimization, distinct RBER values are applied to each sub-page to reflect this mapping.

V. Evaluation

A. Experimental Setup

1) NAND Flash Configuration: Table II summarizes the NAND Flash configurations used in our experiments. We

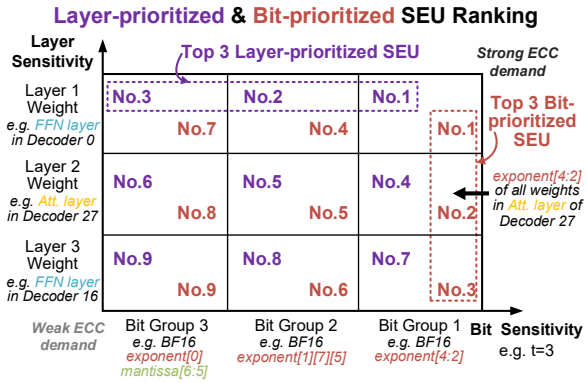

Fig. 8. Layer-prioritized and bit-prioritized SEU ranking.

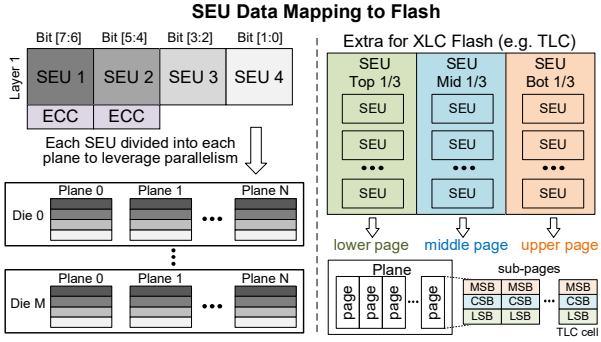

Fig. 9. SEU data mapping to flash and extra consideration for XLC flash.

TABLE II. Flash Configuration

	t_R	Page Size	RBER	Plane Area
SLC	3.5μs	4KB	1e-5	1.702mm^2
TLC	37μs	16KB	0.7/3.5/5e-4	7.388mm^2

adopted SLC and TLC NAND Flash, as employed in Lincoln [7] and AiF[8], respectively. The SLC Flash array configuration is based on XL-Flash [30] and Lincoln's optimizations, conservatively setting the single-plane read latency t_R to 3.5 μs. For TLC, t_R is set to 37μs. To ensure sufficient parallelism, the maximum number of planes is set to 32 [7]. Referring to Lincoln that addresses the reliability of storage and access for large model weights, we conservatively configured flash RBER with a retention time of 1 week and 500 P/E cycles as the conservative baseline, selecting the RBER for SLC and TLC. For TLC, which differentiates between LSB, CSB, and MSB, we applied specific RBER values [28] to reflect its characteristics.

2) Language Model Configuration: For the selection of LLMs, we continued to use the models employed in the error injection experiment. To further ensure the generalizability of our experiments, in addition to the MMLU and Wikitext-2 datasets used in the error injection phase, we incorporated CoLA [31], SST-2 [32], CMMLU [33], RACE [34], and WikiText-103 [25] as additional benchmarks for evaluating the final inference performance.

B. Results

1) LLM Inference Accuracy

We set a 0.5% threshold for accuracy and perplexity degradation. Initially, by injecting errors after fixing high-order exponent bits, we identify the accuracy degradation thresholds across models, as shown in Fig. 3 and Fig. 5. Based on the RBER of SLC and TLC, we then determined the initial Hamming code strength for each model in Table IV. Table III shows that the proposed ECC method preserves LLM

TABLE III. ACCURACY AND PERPLEXITY OF DIFFERENT LLMS WITH BALANCE OPTIMIZATION

		Accuracy					Perplexity	
	Task	CoLA	SST2	MMLU	CMMLU	RACE	Wikitext2	Wikitext103
LLaMA 3-8B	Baseline	68.6	93.5	58.5	52.5	76.2	8.29	5.70
	BALANCE (TLC)	68.3	93.5	58.4	52.1	75.9	8.32	5.71
	BALANCE (SLC)	68.3	93.5	58.3	52.1	76.0	8.32	5.71
OPT-30B	Baseline	60.5	93.2	23.2	24.2	21.5	13.93	14.10
	BALANCE (TLC)	60.3	93.0	23.0	24.1	21.4	14.00	14.15
	BALANCE (SLC)	60.3	93.0	23.1	24.0	21.3	13.99	14.15
LLaMA 3-8B-INT8	Baseline	66.3	93.3	58.0	52.1	75.1	8.31	6.11
	BALANCE (TLC)	66.0	93.3	57.6	52.0	75.0	8.35	6.12
	BALANCE (SLC)	66.1	93.3	57.7	52.1	75.1	8.35	6.12

TABLE IV. HAMMING CODE PROTECTION LEVEL CONFIGURATION AND CODE RATE AFTER PROPOSED GREEDY ALGORITHM

	SLC		TLC	
LLaMA 3-8B	Protection Level	1024bit	Protection Level	32bit
	Code Rate	99.7%	Code Rate	95.5%
OPT-30B	Protection Level	1024bit	Protection Level	16bit
	Code Rate	99.5%	Code Rate	92.8%
LLaMA3-8B-INT8	Protection Level	1024bit	Protection Level	512bit
	Code Rate	99.9%	Code Rate	99.7%

TABLE V. COMPARISON OF ECC OPTIMIZATION

	Lincoln BCH [7]	AiF [8]	LDPC [28]	This Work
Technology	28nm	28nm	28nm	28nm
Area(um²)	289688	64980	630350	1052
Power(μW)	7336	2818	-	8.76
Code Rate	90.1%	90.3%	90.0%	95.5%
Throughput (Gb/s)	14.6	6.4	3.4	6.4

*For a fair comparison, all data are results scaled to a 28nm process

inference accuracy not only on WikiText2 and MMLU but also across other datasets. For example, with LLaMA3-8B, perplexity on WikiText103 increases by only 0.1%, while accuracy drops on CoLA, SST2, MNLI, and RACE are just 0.4%, 0%, 0.1%, and 0.4%, respectively.

2) Comparison of ECC Optimization

For the most ECC-demanding configuration in our evaluation, i.e., the LLaMA3-8B model on TLC NAND. We implemented the ECC circuit at 28nm, with 32-bit SEC Hamming codes, optimized according to the proposed scheme. Each SEU uses a single-bit flag to indicate whether it is protected, keeping hardware overhead minimal despite differentiated protection. Table V compares the area, power, code rate, and throughput of our method with traditional ECC approaches in IFC architectures. For a fair comparison, all data are results scaled to a 28nm process [35].

a) Area: ECC circuit area is 1052μm², reducing the area by 275× compared to Lincoln's BCH code [7], by 62× compared to AiF's BCH code, and by 599× compared to the traditional (9141,8224) LDPC code [36]. Complex LDPC codes are impractical for IFC architectures due to their excessive area, which exceeds the logic die's capacity, while BCH codes also occupy a significant portion of a logic die's area [11].

b) Power: The power consumption is reduced by 899× compared to Lincoln's BCH code, by 321× compared to AiF's BCH code. Benefiting from the simple error correction logic of Hamming codes, BALANCE achieves significant gains in both area and power consumption.

c) Code Rate and Throughput: Our method reduces data redundancy by selectively protecting weights. For the LLaMA3-8B model under TLC configuration, it achieves a code rate of 95.5%, surpassing the previous benchmarks by 6%, 5.7%, and 6.1%, respectively. For the OPT-30B model, the code rate improves by 3%, 2.7%, and 3.1%. In the LLaMA3-int8 quantized model, the code rate increases by 10.6%, 10.4%, and 10.7%. Under the lower RBER SLC configuration, our method proves even more effective, enabling the LLaMA3-8B base model to match the int8 model's code rate. In terms of throughput, our ECC module achieves a processing throughput of 6.4 Gb/s, matching the NAND Flash plane's read bandwidth. This configuration ensures that data transfer to the PE for matrix operations avoids any wastage of read bandwidth due to the ECC module.

3) LLM inference performance improvement

In the IFC system, designed to accelerate LLM inference, reducing the area and power consumption of the ECC component enables the integration of additional PEs on the logic die to perform matrix multiplication. Previous work constrained by the large ECC overhead could only accommodate a limited number of PEs for GEMV operations. However, the increased available area now supports a larger PE array, Lincoln could integrate 2× more PEs, and AiF could integrate 8× more PEs, potentially enabling GEMM operations to assist NPUs. We will leave this possibility for future work to explore.

VI. CONCLUSION

In this work, we proposed BALANCE, a lightweight ECC design for in-flash computing that enhances LLM inference on edge devices. By evaluating bit-level and layer-level significance, our stratified ECC strategy selectively applies Hamming codes and optimizes data placement in NAND Flash. For a LLaMA3-8B model on TLC NAND, our method reduces ECC overhead compared to state-of-the-art BCH codes, achieving up to 275× area and 899× power reductions. This is accomplished while maintaining accuracy degradation within 0.5% and delivering a 6% and 5.7% higher code rate. By minimizing this critical overhead, BALANCE frees up valuable silicon area for processing elements, significantly advancing the deployment of powerful and cost-effective LLMs at the edge.

ACKNOWLEDGMENT

This work was supported in part by the National Natural Science Foundation of China (92464204 & 9246302), in part by the National Key Research and Development Program of China (2022YFB4400600). The corresponding author is Weiwei Shan (wwshan@seu.edu.cn). Changwei Yan and

Lishuo Deng contribute equally to this work. The authors gratefully acknowledge Professor Chen Zhang, Zhongkai Yu, Hongtao Zhong, Yizhe Chen, and Hanjie Liu for their valuable technical suggestions.

REFERENCES

[1] Z. Zhou et al., "A survey on efficient inference for large language models," arXiv, Jul. 2024.

[2] J. Li et al., "Large language model inference acceleration: A comprehensive hardware perspective," arXiv, Oct. 2024.

[3] J. Xu et al., "On-device language models: A comprehensive review," arXiv, Sep. 2024.

[4] G. Heo et al., "NeuPIMs: NPU-PIM heterogeneous acceleration for batched LLM inferencing," in Proc. 29th ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), La Jolla, CA, USA, Apr. 2024, pp. 722–737.

[5] L. Liu et al., "Make LLM inference affordable to everyone: Augmenting GPU memory with NDP-DIMM," in Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA), Mar. 2025, pp. 1751–1765.

[6] Z. Yu et al., "Cambricon-LLM: A chiplet-based hybrid architecture for on-device inference of 70B LLM," in Proc. 57th IEEE/ACM Int. Symp. Microarchitecture (MICRO), Nov. 2024, pp. 1474–1488.

[7] W. Sun et al., "Lincoln: Real-time 50–100B LLM inference on consumer devices with LPDDR-interfaced, compute-enabled flash memory," in Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA), Mar. 2025, pp. 1734–1750.

[8] J. Lee et al., "AiF: Accelerating On-Device LLM Inference Using In-Flash Processing," in Proc. 52nd Annu. Int. Symp. Comput. Archit. (ISCA), Tokyo, Japan, Jun. 2025, pp. 529–543.

[9] A. Vaswani et al., "Attention is all you need," in Adv. Neural Inf. Process. Syst. (NeurIPS), 2017.

[10] C. Li et al., "H2-LLM: Hardware-dataflow co-exploration for heterogeneous hybrid-bonding-based low-batch LLM inference," in Proc. 52nd Annu. Int. Symp. Comput. Archit. (ISCA), Tokyo, Japan, Jun. 2025, pp. 194–210.

[11] M. Chun et al., "PiF: In-flash acceleration for data-intensive applications," in Proc. 14th ACM Workshop Hot Topics Storage File Syst. (HotStorage), Jun. 2022, pp. 106–112.

[12] Z. Huo, W. Cheng, and S. Yang, "Unleash scaling potential of 3D NAND with innovative Xtacking® architecture," in Proc. IEEE Symp. VLSI Technol. Circuits, Jun. 2022, pp. 254–255.

[13] D. B. L. Yolanda, "Wafer to wafer bonding to increase memory density," in Proc. China Semicond. Technol. Int. Conf. (CSTIC), Jun. 2022, pp. 1–4.

[14] W. Liu et al., "Characterizing the reliability and threshold voltage shifting of 3D charge trap NAND flash," in Proc. Design, Autom. Test Europe Conf. Exhib. (DATE), Florence, Italy, Mar. 2019, pp. 312–315.

[15] Y. Cai et al., "Data retention in MLC NAND flash memory: Characterization, optimization, and recovery," in Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA), Feb. 2015, pp. 551–563.

[16] Y. Cai et al., "Error characterization, mitigation, and recovery in flash-memory-based solid-state drives," Proc. IEEE, vol. 105, no. 9, pp. 1666–1704, Sep. 2017.

[17] Y. Luo et al., "HeatWatch: Improving 3D NAND flash memory device reliability by exploiting self-recovery and temperature awareness," in Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA), Feb. 2018, pp. 504–517.

[18] Y. Cai et al., "Read disturb errors in MLC NAND flash memory: Characterization, mitigation, and recovery," in Proc. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN), Jun. 2015, pp. 438–449.

[19] N. Papandreou et al., "Reliability of 3D NAND flash memory with a focus on read voltage calibration from a system aspect," in Proc. IEEE Non-Volatile Memory Technol. Symp. (NVMTS), Oct. 2019, pp. 1–4.

[20] D. Rossi and C. Metra, "Error correcting strategy for high speed and high density reliable flash memories," J. Electron. Test., vol. 19, no. 5, pp. 511–521, Oct. 2003.

[21] X. Zhao et al., "Error bits recovering in 3D NAND flash memory: A novel state-shift re-program (SRP) scheme," in Proc. IEEE Int. Conf. Integr. Circuits Technol. Appl. (ICTA), Oct. 2023, pp. 99–100.

[22] "meta-llama/Meta-Llama-3-8B," accessed Apr. 2025. [Online]. Available: https://huggingface.co/meta-llama/Meta-Llama-3-8B

[23] "facebook/opt-30b," accessed Jul. 2025. [Online]. Available: https://huggingface.co/facebook/opt-30b

[24] "meta-llama/Llama-Guard-3-8B-INT8," accessed Jul. 2025. [Online]. Available: https://huggingface.co/meta-llama/Llama-Guard-3-8B-INT8

[25] S. Merity et al., "Pointer sentinel mixture models," arXiv, Sep. 2016.

[26] D. Hendrycks et al., "Measuring massive multitask language understanding," arXiv, Jan. 2021.

[27] X. Chen et al., "Streamlining redundant layers to compress large language models," arXiv, Jan. 2025.

[28] Y. Deguchi et al., "3-D NAND flash value-aware SSD: Error-tolerant SSD without ECCs for image recognition," IEEE J. Solid-State Circuits, vol. 54, no. 6, pp. 1800–1811, Jun. 2019.

[29] J. Jang and J. H. Ko, "WISER: Deep neural network weight-bit inversion for state error reduction in MLC NAND flash," in Proc. DATE, Feb. 2021, pp. 735–738.

[30] T. Kouchi et al., "A 128Gb 1b/cell 96-word-line-layer 3D flash memory to improve random read latency with tPROG=75µs and tR=4µs," in Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC), Feb. 2020, pp. 226–228.

[31] A. Warstadt et al., "Neural network acceptability judgments," Trans. Assoc. Comput. Linguist., vol. 6, pp. 625–641, 2018.

[32] R. Socher et al., "Recursive deep models for semantic compositionality over a sentiment treebank," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2013, pp. 1631–1642.

[33] H. Li et al., "CMMLU: Measuring massive multitask language understanding in Chinese," arXiv, Jan. 2024.

[34] G. Lai et al., "RACE: Large-scale ReAding comprehension dataset from examinations," arXiv, Dec. 2017.

[35] O. Villa et al., "Scaling the power wall: A path to exascale," in Proc. SC Int. Conf. High Perform. Comput., Nov. 2014, pp. 830–841.

[36] K.-C. Ho et al., "A 3.46 Gb/s (9141,8224) LDPC-based ECC scheme and on-line channel estimation for solid-state drive applications," in Proc. IEEE Int. Symp. Circuits Syst. (ISCAS), Lisbon, Portugal, May 2015, pp. 1450–1453.